SECURITY ASSESSMENT REPORT

yq 4.4.0
DevSecOps Security Report

Assessment of "yq-loyd edition.rar" against the *DevSecOps Checklist (Expanded Edition)*. Includes findings, fixes applied, network-port lockdown and residual risks.

TARGET	yq 4.4.0 (upstream kislyuk/yq, commit `b04512f`)
STANDARD	DevSecOps Checklist (Expanded Edition)
REPORT DATE	2026-10-10
ENVIRONMENT	Python 3.13, PyYAML 6.0.3 with libyaml, jq 1.8.1
TOOLING	Bandit 1.9, zizmor, pip-audit, ruff, mypy, manual review
OVERALL OUTCOME	**Code sound; CI/CD and process gaps fixed.** 0 High, 5 Medium, 6 Low/Info. 1 Medium residual (dependency).

EXECUTIVE SUMMARY

What it is: yq is a Python command-line wrapper that converts YAML, XML or TOML to JSON, pipes it through `jq`, and converts the result back. It is a CLI only: it has no server, no daemon and no network code. The archive is upstream *kislyuk/yq* v4.4.0 (commit `b04512f`); the only difference from git HEAD is Windows CRLF line endings, so no tampering was found.

Verdict: the program code is sound. Most weaknesses sit in the CI/CD and release setup and in missing automated safety nets. One availability bug (crash on extremely deep YAML) lives in the PyYAML/libyaml dependency and could not be safely fixed inside yq. All fixable items were fixed and re-verified.

01 Executive summary

AREA	RESULT
Open network ports	**None found** (static search, import check, tests). A hard-coded, non-removable lockdown was added so none can ever be opened.
Findings	0 High, 5 Medium, 6 Low/Info. Fixed: 4 Medium + 3 Low/Info. Residual: 1 Medium + 3 Low (sections 3 and 6).
Tests	160 original tests pass; 162 pass after changes (2 new network-lockdown tests).
Static scans after fixes	zizmor: 0 findings (was 13 reported). Bandit: 0 medium/high (13 low, accepted). ruff + format: clean. pip-audit on yq dependencies: clean.
Secrets	None in working tree or full git history (regex scan for cloud keys, private keys, tokens, passwords).

02 Scope and method

Tools: Bandit 1.9 (SAST), zizmor (GitHub Actions audit), pip-audit (SCA), ruff and mypy, plus manual code review of all 12 Python modules and the workflows. Dynamic probes were run against the tool (YAML alias bomb, deep nesting, malicious YAML tags, XXE). Environment: Python 3.13, PyYAML 6.0.3 with libyaml, jq 1.8.1. The checklist has 14 sections covering GitHub, cloud and Kubernetes settings; items that are organisation or infrastructure settings (MFA, branch protection, cloud IAM, Kubernetes) cannot be verified from a source archive and are listed as "not verifiable" in section 7.

03 Findings and fixes

F-01 CI/CD supply chain — Checklist 2 [L1] SHA pinning — MEDIUM

FINDING

9 third-party action references used mutable tags (`@v7, @v6, @v5, @v4, @release/v1`) and the two workflows used different major versions of the same action. A moved or hijacked tag would run attacker code in the build and in the PyPI publish job.

FIX / STATUS

Pinned every `uses:` to a full 40-char commit SHA with a version comment (checkout v7.0.1, setup-python v6.3.0, codecov v5.5.5, ruff-action v4.0.0, pypi-publish v1.14.2). Added Dependabot for the github-actions ecosystem so pins are bumped via PR.

F-02 CI/CD token scope — Checklist 2 [L1] Least-privilege tokens — MEDIUM

FINDING

`ci.yml` and `release.yml` had no top-level `permissions:`, so the GITHUB_TOKEN got the repository default, potentially read/write.

FIX / STATUS

Added `permissions: contents: read` at workflow level in all workflows. In release.yml only the publish job adds `id-token: write` (needed for PyPI trusted publishing).

F-03 CI/CD credentials — Checklist 2 [L1] / 4 — LOW

FINDING

`actions/checkout` persisted the git credential in the workspace (zizmor "artipacked"), readable by later steps, including in the release job.

FIX / STATUS

Set `persist-credentials: false` on every checkout.

F-04 Dependency policy — Checklist 3 [L1] SCA — MEDIUM

FINDING

`pyproject.toml` allowed **PyYAML >= 5.3.1**. 5.3.1 is affected by PYSEC-2021-142 (CVE-2020-14343, code execution through the unsafe full loader). yq uses SafeLoader so it is not directly exploitable, but users could install the vulnerable build.

FIX / STATUS

Raised the floor to `PyYAML >= 5.4` (first fixed release). pip-audit of the floor versions of all four runtime dependencies is now clean.

F-05 No automated scanning Checklist 3 [L1], 9 [L1], 5 [L1] — MEDIUM

FINDING
No SCA, SAST, or workflow-lint ran in CI, and no automatic dependency updates existed. New CVEs would go unnoticed.

FIX / STATUS
Added `.github/workflows/security.yml` (Bandit, pip-audit on runtime deps, zizmor), run on push, PR and weekly, SHA-pinned and read-only. Added `.github/dependabot.yml` (actions + pip, weekly, 7-day cooldown against brand-new releases).

F-06 Network exposure User request: close all open ports — INFO

FINDING
No listening or outgoing network code exists: no `socket, http, ssl, urllib` usage in the code, none of those modules are loaded at import, and no inet sockets are used at run time. The only external process is `jq` over pipes.

FIX / STATUS
Added a **hard-coded lockdown** (section 4) so the process can never open or use an IPv4/IPv6 port.

F-07 Repo hygiene Checklist 12 [L1] — INFO

FINDING
All files had CRLF endings, so git reported 33 files as modified and diffs/reviews were unusable.

FIX / STATUS
Added `.gitattributes` (`* text=auto eol=lf`). The delivered tree is clean LF.

F-08 Availability (dependency) Checklist 9 [L3] Fuzzing — MEDIUM

FINDING
YAML nested about 25,000+ levels (for example 30,000 times `[`) makes the PyYAML C composer (CSafeLoader / libyaml) overflow the C stack and **segfault** (exit 139). Reproduced with plain `yaml.load(..., CSafeLoader)`, so the fault is in PyYAML/libyaml. Shallower bombs (1,000 to 20,000 levels) fail cleanly with a RecursionError message. No memory disclosure; one process dies.

FIX / STATUS
Not fixed in yq. A correct fix needs a depth limit inside the parser; the project rules (AGENTS.md) forbid custom pre-parsers, and the pure-Python loader would be much slower. Mitigate by running untrusted input with a timeout and resource limits, and report upstream to PyYAML.

F-09 SAST notes Checklist 9 [L1] — LOW

FINDING
Bandit reported 13 Low items: B404/B603/B607 (subprocess with the partial path `jq`), B101 (asserts used for type narrowing), B110 (try/except/pass when printing jq help).

FIX / STATUS
Reviewed, no exploitable path: arguments are a list (no shell), jq options are by design user-controlled. Left unchanged. Note: `jq` is resolved through PATH, so keep PATH trusted.

F-10 Data integrity Checklist 10 [L2] — LOW

FINDING
`-i/--in-place` rewrites the file directly with no backup and no atomic rename. A crash mid-write leaves a truncated file (the code reports this).

FIX / STATUS
Accepted upstream design (temp files are forbidden by AGENTS.md). Use version control or a copy before using `-i`.

F-11 Support matrix Checklist 3 — LOW

FINDING
Metadata claims Python >= 3.8, but CI tests only 3.10 to 3.15; 3.8/3.9 are end-of-life and untested.

FIX / STATUS
Not changed (maintainer decision). Raise `requires-python` or add those versions to the matrix.

04 Port lockdown ("hard-coded")

Result of the audit: yq opens **no ports**. There is nothing to close, so the safest hardening is to make it structurally impossible.

What was added. New module `yq/_netguard.py`, called as the first action of `cli()` (so yq, xq and tomlq all use it):

- `socket.socket.__init__` is replaced: creating any AF_INET or AF_INET6 socket raises `NetworkAccessBlocked` (a PermissionError). No socket means no bind, listen, accept or connect.
- Helpers `create_server, create_connection` and DNS functions (`getaddrinfo, gethostbyname*, getnameinfo, gethostbyaddr`) are also disabled.
- There is no flag or environment variable to switch it off, and it is idempotent. Unix-domain sockets and pipes (stdin/stdout, jq) still work.
- Library users who call `yq.yq()` directly are not affected; only the command-line entry points lock down.

Verification. Two new tests in a child process confirm that bind, connect, `create_server` and `getaddrinfo` all fail after lockdown, that `socketpair()` still works, and that running `cli()` triggers the lock. The wheel build includes the module.

Limit. This is in-process defence in depth: a native extension could still call the OS directly, and the jq child is not covered. For a hard guarantee also enforce it at OS level, for example `docker run --network none`, a systemd `IPAddressDeny=any`, or a firewall rule.

05 What was tested and passed

CHECK	RESULT
Malicious YAML tag `!!python/object/apply:os.system`	Not executed; treated as plain data (safe loader).
YAML alias / merge-key bomb	Stopped: "detected unsafe YAML entity expansion".
XML external entity (XXE) with `file:///etc/passwd`	Blocked: "entities are disabled".
Very deep XML (50,000 levels)	Clean error message (RecursionError handled), no crash.
Command injection	jq is started with an argument list, no shell.
Secrets in files and git history	None found.
Release publishing	Uses PyPI trusted publishing (OIDC), a protected "release" environment, tag-only trigger.
Full test suite, ruff, ruff format	Pass before and after.

06 Residual risks and recommended next steps

- **Medium:** deep-nesting segfault (F-08): apply timeouts/resource limits for untrusted input; file an upstream PyYAML/libyaml issue.
- Commit a lockfile with hashes for CI (`pip install --require-hashes`) and pin `pip install build` in release.yml (Checklist 3 [L1]).
- Add CODEOWNERS for `.github/workflows/` and `pyproject.toml`, and require review (Checklist 1 [L1]).
- Generate an SBOM and sign release artifacts with Cosign/attestations (Checklist 2 [L2/L3]).
- Add continuous fuzzing of the YAML/XML/TOML input paths (Checklist 9 [L3]).
- Decide the supported Python range (F-11) and add `types-xmltodict` to the test extras so mypy runs cleanly without network install.

07 Checklist coverage

CHECKLIST SECTION	STATUS FOR THIS PROJECT
1 Source control and identity	Not verifiable from an archive (MFA, branch protection, signed commits are platform settings). CODEOWNERS missing: recommended.
2 CI/CD and supply chain	**Fixed:** SHA pinning, read-only tokens, no persisted credentials, OIDC publish retained. Open: SBOM, provenance (L2/L3). No `pull_request_target` or untrusted-input interpolation found.

CHECKLIST SECTION	STATUS FOR THIS PROJECT
3 Dependencies	**Improved:** PyYAML floor, Dependabot, pip-audit in CI. Open: lockfile, internal proxy, license policy.
4 Secrets	No secrets present. Push protection and pre-commit scanning are platform/dev settings: recommended (Gitleaks).
5 IaC	Not applicable (no Terraform/Kubernetes/Helm). Workflows linted by zizmor.
6 and 7 Containers and Kubernetes	Not applicable (no Dockerfile or manifests). The suggested `--network none` run mode follows these principles.
8 Cloud IAM and data	Not applicable.
9 Security testing	**Added** SAST (Bandit) in CI; dynamic probes done manually; fuzzing recommended.
10 and 11 Detection, vulnerability management	Weekly scheduled scan added; findings tracking and SLAs are organisational.
12 and 13 DevEx, governance	LF normalisation and automation added; reviews, evidence and metrics are organisational.

08 Files changed

FILE	CHANGE
`yq/_netguard.py (new)`	Hard-coded IPv4/IPv6 lockdown
`yq/__init__.py`	Import and call lock_down_network() at start of cli()
`test/test.py`	2 new lockdown tests
`pyproject.toml`	PyYAML >= 5.4
`.github/workflows/ci.yml`	SHA pins, permissions, persist-credentials false
`.github/workflows/release.yml`	SHA pins, least-privilege permissions, persist-credentials false
`.github/workflows/security.yml (new)`	Bandit, pip-audit, zizmor; weekly
`.github/dependabot.yml (new)`	Weekly updates with 7-day cooldown
`.gitattributes (new)`	Force LF line endings